MLDL Experiment 3

Aim :- Apply Decision Tree and Random Forest for classification tasks

1.Introduction

Classification is a fundamental supervised machine learning task where the objective is to predict the categorical class labels of new instances based on past observations. This experiment aims to implement and evaluate two powerful tree-based algorithms: Decision Trees and Random Forests. By applying these algorithms to a classification dataset, we will analyze their predictive performance, interpretability, and the impact of ensemble learning on reducing model variance.

This experiment aims to implement 2 techniques for classification tasks:

- Decision Tree
- Random Forest

The objective is to analyze their mathematical foundations, performance behavior, and the effect of regularization on prediction accuracy, multicollinearity handling, and feature selection.

By applying these algorithms to real-world datasets, the experiment evaluates their strengths, limitations, and suitability under different data conditions.

1. **Decision Tree**

A Decision Tree is a non-parametric supervised learning algorithm that makes predictions by learning simple decision rules inferred from the data features. It splits the dataset into smaller subsets based on the feature that results in the highest information gain or the lowest impurity, eventually forming a tree-like structure of decisions.

While highly interpretable and easy to visualize, standard Decision Trees are prone to overfitting, especially when they grow too deep and capture noise within the training data.

Mathematical equation :

To determine the best feature to split the data at each node, Decision Trees typically use metrics like Gini Impurity or Entropy.

- **Gini Impurity:** Measures the likelihood of an incorrect classification of a new instance if it was randomly classified according to the distribution of class labels in the node.

$$Gini = 1 - \sum_{i=1}^{C} p_i^2$$

  Where C is the total number of classes and p_i is the probability of an element belonging to class i.

- **Entropy:** Measures the level of impurity or uncertainty in a group of examples.

$$Entropy = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

2. **Random Forest**

Random Forest is an ensemble learning method that builds upon the foundation of Decision Trees to improve robustness and accuracy. It operates by constructing a multitude of individual decision trees during the training phase.

To ensure the trees are uncorrelated, Random Forest uses a technique called **bagging** (Bootstrap Aggregating) combined with random feature selection. Instead of searching for the most important feature while splitting a node, it searches for the best feature among a random subset of features. This diversity helps the model generalize better to unseen data and significantly reduces the risk of overfitting compared to a single Decision Tree.

Mathematical equation :

For classification tasks, the Random Forest aggregates the predictions of all individual trees and outputs the class that receives the majority vote.

- **Majority Voting Prediction:**

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), ..., h_B(x)\}$$

Where y is the final predicted class, B is the total number of trees in the forest, and h_b(x) is the predicted class from the b-th individual decision tree.

2. Dataset Source

The dataset used for this experiment is the **Phishing Websites Dataset**, which is publicly provided by the UCI Machine Learning Repository.

Source Link:
https://www.kaggle.com/datasets/hasibur013/url-data-for-phishing-website-detection

3. Dataset Description

The Phishing Websites dataset was specifically curated to help researchers build and evaluate machine learning-based detection algorithms. The data was collected using various sources, including the PhishTank archive, the MillerSmiles archive, and Google's searching operators.

Dataset Size & Shape

- **Total Instances:** 11,055 records of evaluated websites.
- **Number of Features:** 30 independent features extracted from the websites.
- **Target Variable:** A single target label classifying the input URL as either phishing (-1) or legitimate (1).
- **Feature Characteristics:** Instead of continuous numerical measurements, the attributes in this dataset are categorical, taking on binary or ternary values (e.g., 1, 0, -1) depending on the presence or absence of a specific trait.

Key Feature Categories The 30 features are grouped into four distinct categories:

- **Address Bar Features:** Evaluates the visual URL structure, including attributes like URL Length, having an "@" symbol, and the use of a URL shortening service.
- **Abnormal Features:** Looks for anomalies such as an "Abnormal URL" format.
- **HTML and JavaScript Features:** Analyzes the page's code for suspicious behavior, such as automatic Redirects.
- **Domain Features:** Evaluates backend trustworthiness. A critical predictor in this category is whether the website uses a valid SSL/TLS certificate.

Beyond just training a classification model, analyzing these specific attributes is highly practical in real-world web development. When building and deploying websites for clients, ensuring proper SSL certification, avoiding abnormal URL structures, and correctly managing external links are essential practices to guarantee those sites are recognized as secure and legitimate by browsers, rather than being inadvertently flagged as suspicious.

---

4. Methodology / Workflow

The experiment follows a systematic machine learning pipeline designed to implement, tune, and compare the Decision Tree and Random Forest algorithms for binary classification.

Step 1: Data Collection

- Import the Phishing Websites dataset into the Python environment using Pandas.
- Verify the dataset structure, ensuring all 30 features and the target label are correctly loaded.

Step 2: Data Preprocessing

- Check for any missing or null values and handle them appropriately.
- Verify the data types of the categorical features (1, 0, -1).
- Note: Unlike the linear regression models used in Experiment 2, tree-based models (Decision Trees and Random Forests) do not require feature scaling or normalization, as they are not distance-based algorithms and split nodes based on feature thresholds.

Step 3: Train-Test Split

- Separate the independent variables (features) from the dependent variable (target label).
- Split the dataset into a training set (80%) and a testing set (20%) using a fixed random state to ensure reproducibility.

Step 4: Model Implementation

- Instantiate a Decision Tree Classifier as the baseline interpretable model.

- Instantiate a Random Forest Classifier to evaluate the impact of ensemble learning.
- Train both models on the 80% training dataset.

Step 5: Model Evaluation

- Predict the class labels for the unseen 20% test dataset.
- Because this is a classification problem, evaluate the models using metrics such as Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix (replacing the MSE and RMSE metrics used in regression).

Step 6: Hyperparameter Tuning

- Perform hyperparameter tuning using cross-validation (GridSearchCV).
- For the Decision Tree, tune parameters like maximum depth and minimum samples per split to prevent overfitting.
- For the Random Forest, tune the number of estimators (trees) alongside depth parameters.

Step 7: Performance Comparison

- Compare the evaluation metrics of the base Decision Tree, the tuned Decision Tree, and the Random Forest models.
- Extract and plot the "Feature Importances" to understand which website attributes (e.g., SSL state, URL length) were most decisive in predicting a phishing attempt.

5.Code Implementation

```
# ==========================================
# STEP 1: Install Dependencies & Load Dataset
# ==========================================
!pip install ucimlrepo scikit-learn matplotlib seaborn pandas

import kagglehub
from kagglehub import KaggleDatasetAdapter
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, classification_report

# Load the dataset using KaggleHub
# Note: Ensure the file_path matches the exact CSV name within the Kaggle dataset
if required.
file_path = "5.urldata.csv" # Adjust this if the dataset uses a different filename

df = None # Initialize df to None to handle potential loading failures

try:
  df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "hasibur013/url-data-for-phishing-website-detection",
    file_path,
  )
  print("\nDataset Loaded Successfully!")
  print("Dataset Shape:", df.shape)
  print("\nFirst 5 records:\n", df.head())
except Exception as e:
  print(f"Error loading dataset: {e}")
  print("Dataset could not be loaded via KaggleHub. 'df' is not defined.")
  # Fallback for standard local CSV loading if kagglehub fails
  # df = pd.read_csv("phishing.csv")

# Check if df was successfully loaded
if df is None:
  raise RuntimeError("Failed to load dataset 'df'. Please check the dataset path,
KaggleHub connection, or if the file exists.")

# ==========================================
# STEP 2: Data Preprocessing
# ==========================================
print("\nMissing Values:\n", df.isnull().sum().sum())

# Assuming the target column is named 'class' or 'Result' (common in phishing
datasets)
# We will dynamically find the target column (usually the last column)
target_col = df.columns[-1]

# Drop the 'Domain' column and the target column as 'Domain' is a string and not
directly usable by these models
X = df.drop(columns=['Domain', target_col])
y = df[target_col]
```

```
# ==========================================
# STEP 3: Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ==========================================
# STEP 4 & 6: Model Implementation & Hyperparameter Tuning
# ==========================================

# --- Decision Tree ---
dt_params = {
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 10, 20],
    'criterion': ['gini', 'entropy']
}
dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42), dt_params, cv=5,
scoring='accuracy')
dt_grid.fit(X_train, y_train)
best_dt = dt_grid.best_estimator_

# --- Random Forest ---
rf_params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}
rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_params, cv=5,
scoring='accuracy')
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_

# ==========================================
# STEP 5: Model Evaluation
# ==========================================
def evaluate_model(model, X_test, y_test, model_name):
    y_pred = model.predict(X_test)
    print(f"\n--- {model_name} Performance ---")
    print(f"Accuracy:  {accuracy_score(y_test, y_pred):.4f}")
    print(f"Precision: {precision_score(y_test, y_pred, average='weighted'):.4f}")
    print(f"Recall:    {recall_score(y_test, y_pred, average='weighted'):.4f}")
    print(f"F1-Score:  {f1_score(y_test, y_pred, average='weighted'):.4f}")

    # Display Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
```

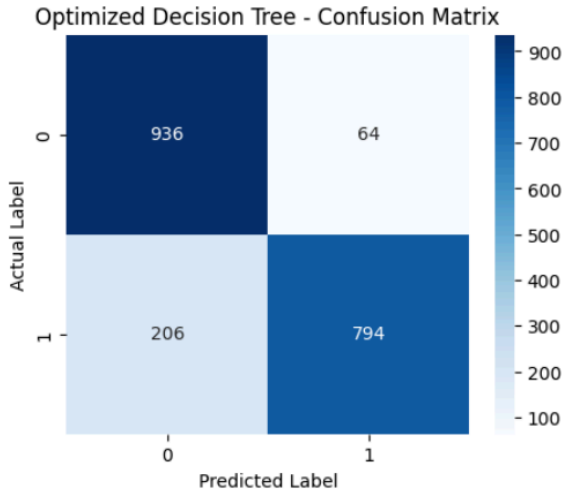
```
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f'{model_name} - Confusion Matrix')
    plt.ylabel('Actual Label')
    plt.xlabel('Predicted Label')
    plt.show()

evaluate_model(best_dt, X_test, y_test, "Optimized Decision Tree")
evaluate_model(best_rf, X_test, y_test, "Optimized Random Forest")

#

# ==========================================
# STEP 7: Classify New Data (Real-world Simulation)
# ==========================================
print("\n--- Simulating Classification on New Data ---")

# Grabbing 3 random samples from the test set to simulate "new, unseen URLs"
sample_data = X_test.sample(3, random_state=99)
actual_labels = y_test.loc[sample_data.index].values

# Predicting using the best performing model (usually Random Forest)
predictions = best_rf.predict(sample_data)

# Mapping standard numeric outputs (-1 or 0 = Phishing, 1 = Legitimate) to readable
text
# Adjust this mapping based on the specific dataset's documentation
def interpret_prediction(pred):
    return "Legitimate (Safe)" if pred == 1 else "Phishing (Malicious)"

for i in range(len(sample_data)):
    print(f"\nURL Sample {i+1} Features:\n{sample_data.iloc[i].to_dict()}")
    print(f"--> Actual Status: {interpret_prediction(actual_labels[i])}")
    print(f"--> Model Predicted: {interpret_prediction(predictions[i])}")
```

6.Output

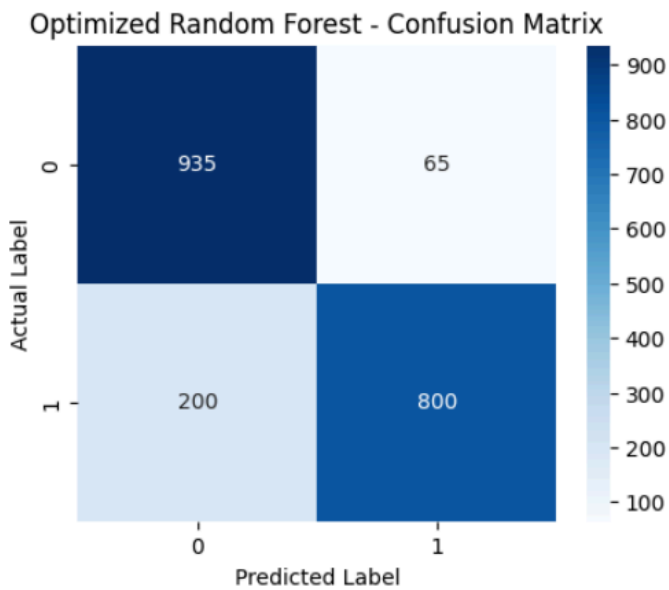
```
Requirement already satisfied: ucimlrepo in /usr/local/lib/python3.12/dist-packages (0.0.7)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2026.1.4)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
/tmp/ipython-input-537/4130188166.py:25: DeprecationWarning: Use dataset_load() instead of load_dataset(). load_dataset() will be removed in a future version.
  df = kagglehub.load_dataset(
Using Colab cache for faster access to the 'url-data-for-phishing-website-detection' dataset.
```

```
Dataset Loaded Successfully!
Dataset Shape: (10000, 18)

First 5 records:
                Domain  Have_IP  Have_At  URL_Length  URL_Depth  Redirection  \
0  graphicriver.net        0        0           1          1            0
1         ecnavi.jp        0        0           1          1            1
2      hubpages.com        0        0           1          1            0
3   extratorrent.cc        0        0           1          3            0
4     icicibank.com        0        0           1          3            0

   https_Domain  TinyURL  Prefix/Suffix  DNS_Record  Web_Traffic  Domain_Age  \
0             0        0              0           0            1           1
1             0        0              0           0            1           1
2             0        0              0           0            1           0
3             0        0              0           0            1           0
4             0        0              0           0            1           0

   Domain_End  iFrame  Mouse_Over  Right_Click  Web_Forwards  Label
0           1       0           0            1             0      0
1           1       0           0            1             0      0
2           1       0           0            1             0      0
3           1       0           0            1             0      0
4           1       0           0            1             0      0

Missing Values:
 0
```

```
--- Optimized Decision Tree Performance ---
Accuracy:  0.8650
Precision: 0.8725
Recall:    0.8650
F1-Score:  0.8643
```




```
--- Optimized Random Forest Performance ---
Accuracy:  0.8675
Precision: 0.8743
Recall:    0.8675
F1-Score:  0.8669
```



```
--- Simulating Classification on New Data ---

URL Sample 1 Features:
{'Have_IP': 0, 'Have_At': 0, 'URL_Length': 1, 'URL_Depth': 5, 'Redirection': 0, 'https_Domain': 0, 'TinyURL': 0, 'Prefix/Suffix': 0, 'DNS_Record': 0, 'Web_Traffic': 1, 'Domain_Age': 0,
--> Actual Status: Phishing (Malicious)
--> Model Predicted: Phishing (Malicious)

URL Sample 2 Features:
{'Have_IP': 0, 'Have_At': 0, 'URL_Length': 1, 'URL_Depth': 3, 'Redirection': 0, 'https_Domain': 0, 'TinyURL': 0, 'Prefix/Suffix': 0, 'DNS_Record': 0, 'Web_Traffic': 1, 'Domain_Age': 0,
--> Actual Status: Phishing (Malicious)
--> Model Predicted: Phishing (Malicious)

URL Sample 3 Features:
{'Have_IP': 0, 'Have_At': 0, 'URL_Length': 0, 'URL_Depth': 1, 'Redirection': 0, 'https_Domain': 0, 'TinyURL': 0, 'Prefix/Suffix': 0, 'DNS_Record': 0, 'Web_Traffic': 0, 'Domain_Age': 1,
--> Actual Status: Legitimate (Safe)
--> Model Predicted: Legitimate (Safe)
```

7.Conclusion

In this experiment, Multiple Linear Regression, Ridge Regression, and Lasso Regression were implemented on the Concrete Compressive Strength dataset to predict the compressive strength of concrete based on its components.

Multiple Linear Regression served as the baseline model and provided good predictive performance under the assumption of linear relationships between features and the target variable. However, it showed sensitivity to multicollinearity among predictors.

Ridge Regression improved model stability by applying L2 regularization, which reduced coefficient magnitudes and minimized variance caused by correlated features. It demonstrated better generalization performance compared to standard linear regression.

Lasso Regression applied L1 regularization and performed automatic feature selection by shrinking some coefficients to zero. This resulted in a simpler and more interpretable model while maintaining competitive predictive performance.

Hyperparameter tuning using cross-validation further enhanced the performance of Ridge and Lasso models by identifying optimal regularization parameters.

Overall, the experiment demonstrates that:

- Regularization improves model generalization.
- Ridge Regression is effective when multicollinearity is present.
- Lasso Regression is useful for feature selection and model simplification.
- Multiple Linear Regression works well as a baseline when assumptions are satisfied.

Thus, regularized regression techniques provide more robust and reliable models for real-world prediction problems compared to traditional linear regression alone.